

5. Analytics Approach

*“The goal is to turn data into information,
and information into insight.”*

— Carly Fiorina

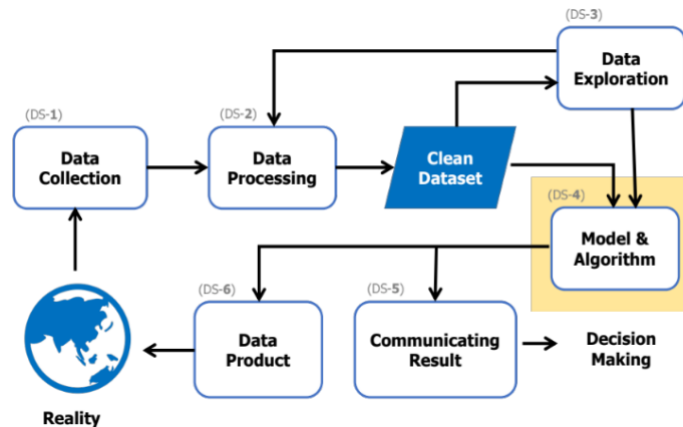


Figure 5.0-1: Model development in the data science process

This chapter covers how to build a model — the Model & Algorithm step (DS-4), a central stage of the analytics process (Figure 5.0-1). It covers what a model is, how to choose a technique, how to evaluate a model, and the model-building process, so the resulting model is trustworthy and production-ready. This chapter overviews supervised learning at the predictive-analytics level — a branch of machine learning, itself a branch of artificial intelligence — making it a foundational chapter on the path to becoming a data scientist.

5.1 Model

A model, in this book, means a mathematical model — a function built from mathematical equations, refined through technique and learning from historical data, ready to be implemented in code to compute results from new input.

For example, the famous mass–energy equation $E=mc^2$, discovered by Albert Einstein, can be written as a model function:

$$f(m) = 8.98755179 \times 10^{16} \times m$$

where $f(m)$ returns energy in joules, m is mass in kilograms, and $8.98755179 \times 10^{16}$ is the speed of light squared — a constant multiplied by mass to yield energy. Since mass alone determines energy through this equation, with nothing left to learn or adjust, this equation is a complete model, and can be implemented directly in Python:

```
def f(m):
    return m*8.98755179e16
```

Now imagine Einstein hadn't yet finished this discovery, and had only proposed the structure:

$$f(m) = a \times m$$

where a is some constant left for future physicists to determine experimentally. This isn't yet a model, since the function can't produce a result even with m known, because a is unknown. We might call $f(m)=a \times m$ a technique — say, “the mass–energy relation.” If a later physicist determines $a=1.234$ from experiments at the North Pole, that becomes a model for converting mass to energy at the North Pole; if another finds $a=0.123$ from desert experiments, that becomes a model for the desert instead.

In short: a model is a function — a mathematical equation — that can be implemented directly to produce results from given inputs, such as $f(x)=8.98755179 \times 10^{16} \times m$. A mere equation structure or idea, like the mass–energy relation $f(m)=a \times m$ without a known value for a , is only a technique or concept, not yet a model.

An algorithm, meanwhile, is a process or approach that uses data structures, logic, and a sequence of operations, together with a mathematical model, to compute a result from input data. Here, an algorithm's key role is finding a model's constants — e.g., finding the right value of a in $f(m)=a \times m$ to complete the equation into a usable model. In data analytics, one job of an algorithm is producing an analytics model through learning from data. Two example techniques illustrate this:

Linear Regression

Equation structure:

$$f(X) = b_0 + b_1x_1 + b_2x_2 + \dots$$

Technique/algorithm used to build the model: e.g., Gradient Descent (GD)

Example model:

$$f(x_1, x_2) = 5 - 3x_1 + 20x_2$$

Decision Tree

Equation structure, as a rule set:

$$f(X) = \begin{cases} y_1, & \text{condition on some of } X \\ y_2, & \text{condition on some of } X \\ \dots & \dots \\ \dots & \dots \end{cases}$$

Technique/algorithm used to build the model: e.g., ID3

Example model, as a rule set:

$$f(w, h) = \begin{cases} car, & (w = 4) \wedge (h < 2) \\ truck, & (w = 4) \wedge (h \geq 4) \end{cases}$$

5.2 Choosing a Technique for Model Building

Data analytics offers many techniques to choose from — Linear Regression, Logistic Regression, Decision Tree, Naïve Bayes, Artificial Neural Network, and more (Bishop & Nasrabadi, 2006). Given this variety, analysts must clearly understand the problem at hand to choose the right technique and evaluation method.

Analytics techniques fall under artificial intelligence (AI), which splits into two branches: Rule-Driven AI and Data-Driven AI (Figure 5.2-1). Rule-Driven AI, the earlier approach, relies on experts designing models from their own experience, then implementing them in code. As computing technology has advanced — collecting more data, more variables, beyond what humans can feasibly analyze by hand — Data-Driven AI has surged ahead, letting computers build models by learning from data, a key advance underlying machine learning. This book focuses on Data-Driven AI techniques — chiefly, machine learning.

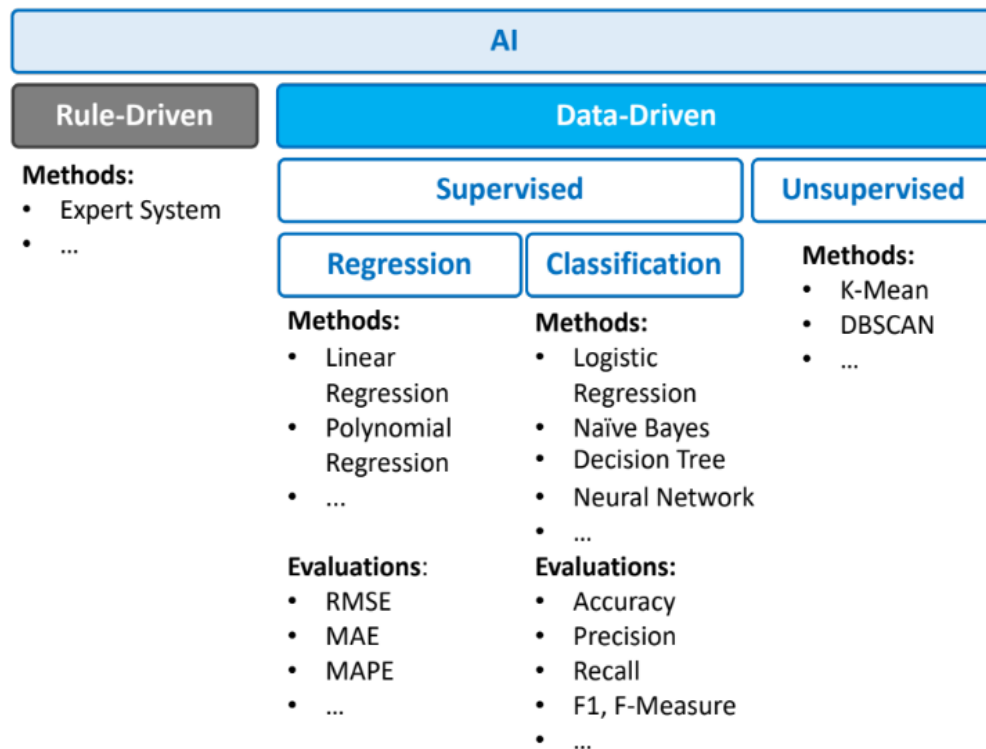


Figure 5.2-1: Structuring an analytics problem

Data-driven AI (machine learning) requires learning from data to build a model, split into two categories: Supervised Learning and Unsupervised Learning — “supervision” here meaning the presence of output/target data.

Supervised Learning

Since “supervision” means having output data, supervised learning means learning from a dataset with both input and output data. Techniques in this category train themselves — adjusting equation constants — so the model's predicted output matches the actual output as closely as possible.

Once a problem is confirmed as supervised learning, the next step is determining whether it's a Regression Analysis or Classification Analysis problem, based mainly on the target variable's data scale.

Regression Analysis

If the target to predict is interval- or ratio-scale (continuous numbers), it's a regression problem. Popular techniques include Linear Regression and Polynomial Regression, plus time-series techniques like Exponential Smoothing or Long Short-Term Memory (LSTM) deep learning. Evaluation typically uses error measures such as RMSE (Root Mean Square Error), MAE (Mean Absolute Error), or MAPE (Mean Absolute Percent Error).

Ordinal-scale data (ordered categories) can also use regression, but must first be encoded into ordered numbers — a challenging step, since ordinal data doesn't specify the gap between ranks. This requires careful encoding expertise, predicting a continuous value, then converting the predicted number back into ordinal categories at the end.

Classification Analysis

If the target to predict is nominal-scale (unordered categories), it's a classification problem, producing a category name (not a continuous number) as output. Common techniques include Logistic Regression, Naïve Bayes, Decision Tree, K-Nearest Neighbors, and Artificial Neural Network. Popular evaluation methods include Accuracy, Precision, Recall, and F1 (F-Measure).

Ordinal-scale data can also use classification, but classification evaluation only judges right or wrong — e.g., for shirt size, if the true value is S but the model predicts M or XL, both count as equally wrong, even though M is intuitively closer. If measuring the degree of error matters, regression-style evaluation can be used instead, after first encoding the ordinal scale into numbers.

Unsupervised Learning

Unsupervised learning has only input data, giving the algorithm a central role in producing results. Most such analysis is cluster analysis — grouping similar items together — using techniques like K-Means or DBSCAN. Results here can't be judged right or wrong, only whether the grouping is mathematically reasonable (per clustering-evaluation metrics) and acceptable to the problem's stakeholders.

Table 5.2-1 summarizes this: the analytics problem, the target's data scale, example techniques, and example evaluation methods — a guide for framing a problem correctly and choosing the right techniques and evaluation approach. Techniques and evaluation methods are covered in detail in later chapters.

Table 5.2-1: Summary guide for framing analytics problems

Analytics Problem	Target Data Scale	Example Techniques	Example Evaluation
Regression Analysis	- Interval scale - Ratio scale - Ordinal scale (encoded)	- Linear Regression - Polynomial Regression - Exponential Smoothing - LSTM	- RMSE - MAE - MAPE
Classification Analysis	- Nominal scale - Ordinal scale (evaluation-dependent)	- Logistic Regression - Naïve Bayes - Decision Tree - K-Nearest Neighbors - Artificial Neural Network	- Accuracy - Precision - Recall - F1 (F-Measure)
Cluster Analysis	Groups (based on input-feature similarity)	- K-Means - DBSCAN	- Stakeholder acceptance - Cohesion measures, e.g., SSE

5.3 Approaches to Evaluating Models

Once a model is built, its performance must be tested — how accurate is it? This applies to supervised learning, since real (actual) output exists to compare against predicted output. There are three approaches:

1. Train and test on the same dataset.
2. Train-Test Split — split data into two sets, e.g., 70% for training, the rest for testing.
3. K-Fold Cross-Validation — similar to Train-Test Split, but split into K equal parts, testing on each part in turn.

In practice, a model is built for future use — meaning it will face data it hasn't seen yet. So approach 1 (train and test on the same data) is unrealistic, like giving students the exact exam questions they just practiced with. In data analytics and machine learning practice, approaches 2 and 3 — Train-Test Split and K-Fold Cross-Validation — are therefore preferred, explained next.

Train-Test Split

Train-Test Split divides data into two sets: one for training, one for testing, as in Figure 5.3-1.

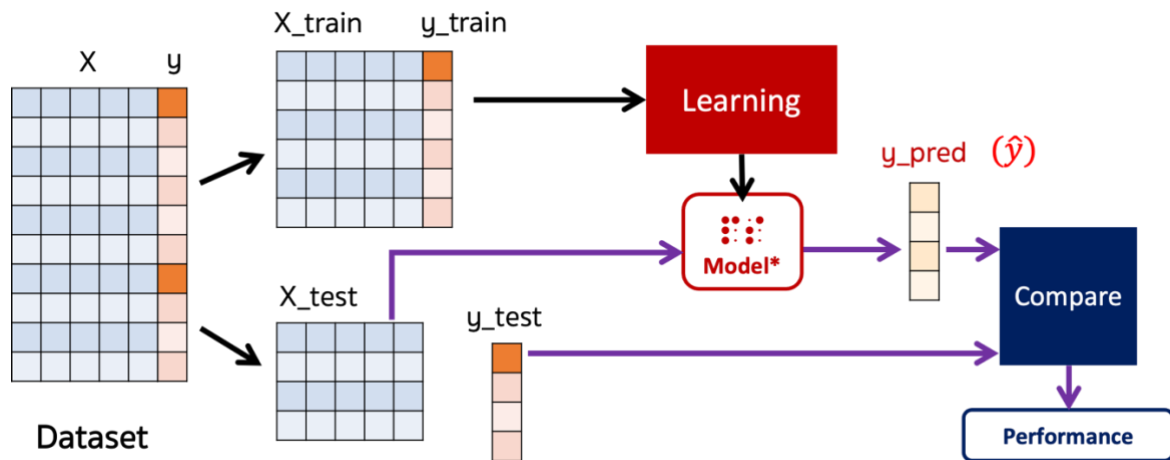


Figure 5.3-1: Approach to splitting data for testing

A dataset typically has several input variables (also called attributes or features) and one output variable (also called a label). In a table or DataFrame, input variables are the various columns, and the output is a single column; the input set is called X (capital X), made up of individual inputs x (lowercase x), and the single output is called y (lowercase y).

The data is split into a Training set (X_{train} , y_{train}) and a Test set (X_{test} , y_{test}). Training uses X_{train} and y_{train} to produce a temporary model (Model^*) — built purely for testing, not for production use.

This temporary model then predicts outputs for the test inputs (X_{test}), producing predicted outputs (y_{pred} , i.e., “predicted y ,” or $\hat{y}$, read “ y -hat”).

The final step compares the predicted output (y_{pred}) against the actual test output (y_{test}), using an evaluation method suited to the learning type — e.g., RMSE for regression, or Accuracy for classification.

Remember that this process builds only a temporary model, trained on a subset of the data. Once you're confident the chosen technique performs well, retrain it on the entire dataset to build the model used in production.

While Train-Test Split feels realistic and is widely accepted academically and in practice, results depend on how the data happens to be randomly split — sometimes yielding a good result, sometimes not. This motivates the next approach: multiple rounds of testing.

K-Fold Cross-Validation

K-Fold Cross-Validation splits data into K parts and tests K times — e.g., splitting into 3 parts ($K=3$) is called 3-Fold Cross-Validation, testing on each of the 3 parts in turn. This is more reliable, since every data point gets tested at some point, as in Figure 5.3-2.

Suppose a dataset has 300 rows. Using 3-Fold Cross-Validation splits it into 3 equal parts — $C1$, $C2$, $C3$ (100 rows each), then trains 3 temporary models:

4. Round 1: train on 200 rows from $C2$ and $C3$, producing temporary model 1; test on the 100 rows in $C1$, giving evaluation score $e1$.
5. Round 2: train on 200 rows from $C1$ and $C3$, producing temporary model 2; test on the 100 rows in $C2$, giving evaluation score $e2$.
6. Round 3: train on 200 rows from $C1$ and $C2$, producing temporary model 3; test on the 100 rows in $C3$, giving evaluation score $e3$.

The final performance score is the average of $e1$, $e2$, and $e3$.

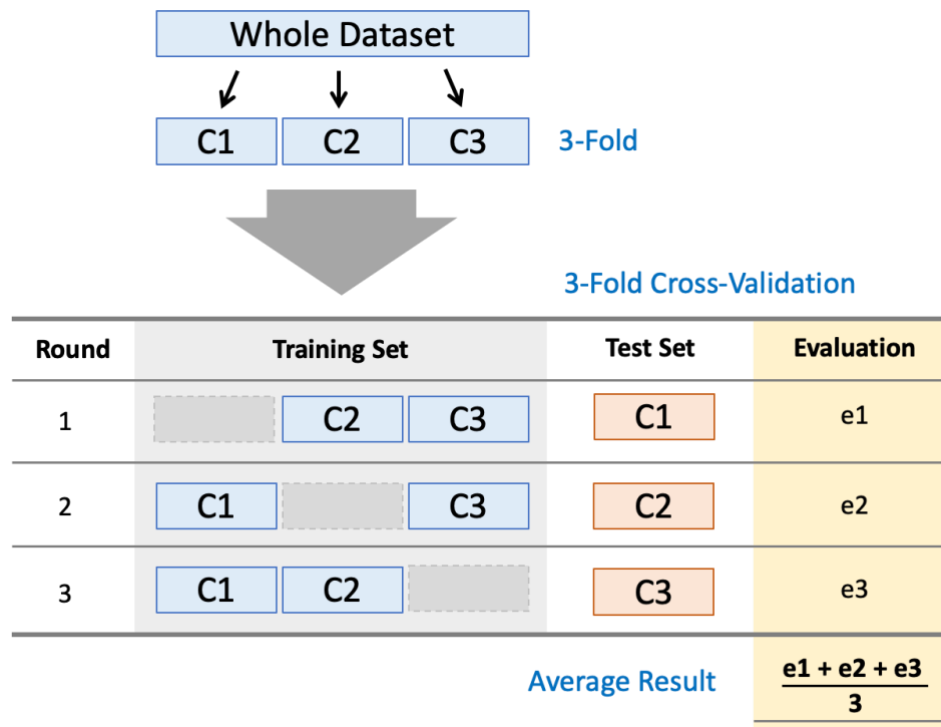


Figure 5.3-2: K-Fold Cross-Validation (K=3)

In practice, K is commonly 3, 4, 5, or 10 — higher K means less test data per round but more rounds, so analysts should choose K based on having enough training data per round while keeping a reasonable test-set size, and should record the K value used when reporting results.

As before, remember each round's model is temporary, for testing only. Once satisfied with a technique's results, retrain it on the full dataset for production use.

While more reliable than Train-Test Split, K-Fold Cross-Validation costs K times the computation time — for slow-to-train models (e.g., deep learning on large datasets), Train-Test Split may be the more practical choice instead.

5.4 The Model-Building Process

Having covered model evaluation, this section details the full model-building process and defines the experimental environment used throughout it.

Experimental Environment (ENV)

Before the model-building process itself, this section defines the Experimental Environment (ENV) as the relation:

$$env = (t, A, F)$$

where t is the chosen analytics technique, A is a set of parameter-argument pairs configuring technique t, and F is the set of features (input variables/attributes) used in this environment. For example, if an experimental environment is:

(logistic_regression, { (max_iter, 100), (tol, 0.01) }, { "weight", "height", "salary" })

this means: the technique is Logistic Regression, with two argument settings — max_iter=100 and tol=0.01 — and the feature set is weight, height, and salary as inputs.

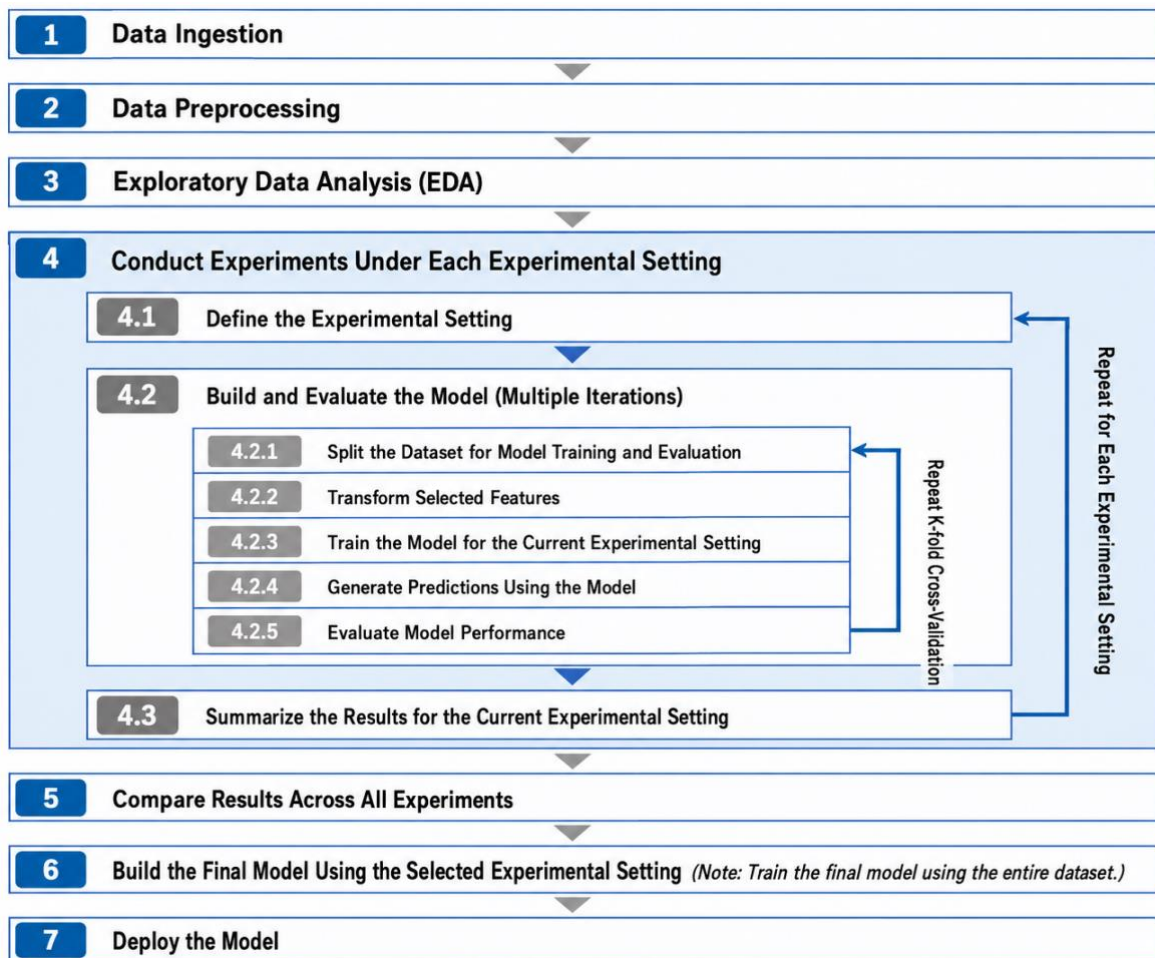


Figure 5.4-1: The model-building process

Steps in the Model-Building Process

This book's model-building process has 7 main steps, with sub-steps, as summarized in Figure 5.4-1 and described below. Each step applies to both regression and classification analysis and can be used directly as a reference for writing code.

1. Import Data

Import structured, ready-to-use data, e.g., a clearly structured table.

2. Basic Data Processing

If the data isn't fully ready — missing data, abnormal values, or unsuitable structure — handle it here: missing-value handling, outlier handling, or transforming data to suit the intended technique. Skip this step if already done earlier.

3. Check Data Behavior

Check each variable's data type, range, and distribution, to guide technique selection or send data back for further processing. Skip this step if already done earlier.

4. Run Experiments for Each Experimental Environment

This step covers multiple experiments, each with its own experimental environment — a chosen technique, parameter settings, and feature set — varied according to the analyst's strategy. Sub-steps:

4.1 Define the Experimental Environment

Define env by choosing the analytics technique, setting each parameter's argument values, and selecting features from the dataset — a strategic decision the analyst makes based on the problem and the data available.

4.2 Build and Test the Model (Multiple Rounds)

Build and test the model via K-Fold Cross-Validation (with the following sub-steps per round), or via Train-Test Split (a single round with the same sub-steps):

1. Split data into training and test sets — Training set (X_{train} , y_{train}) and Test set (X_{test} , y_{test}).
2. Transform selected features if needed — e.g., fit a scaler on X_{train} , then apply it to transform both X_{train} and X_{test} .
3. Build a temporary model per the chosen experimental environment, training on X_{train} and y_{train} .
4. Predict outputs — apply the temporary model to X_{test} , producing predicted outputs y_{pred} .
5. Evaluate the round — compare y_{pred} against the actual y_{test} to score performance.

4.3 Summarize This Environment's Results

Average the per-round results from step 4.2.5 (across all cross-validation rounds) to get this environment's overall test result.

5. Compare Results Across Experiments

Compare the summarized results (step 4.3) across all experimental environments tried.

6. Build the Final Model from the Chosen Environment

Select whichever experimental environment gave the best or most satisfactory result (or per the analyst's judgment), then use its technique, parameter arguments, and feature set to build the production model, trained on the entire dataset.

7. Deploy the Model

Deploy the model into a real application, to predict outcomes for future incoming data.

This process provides a reliable, science-driven path to an effective model, and will be referenced throughout the following chapters — though some steps may be trimmed or adapted depending on the problem or data at hand.

5.5 Chapter Summary

This chapter covered the analytics approach: what a model means here — a mathematical model built by learning from data via analytics techniques and algorithms, ready to compute results from given inputs — followed by choosing between supervised and unsupervised learning. Supervised learning splits into regression (predicting continuous values) and classification (predicting categories), so recognizing which type a problem falls under is essential for choosing the right technique and evaluation method. The chapter then covered model evaluation via Train-Test Split and K-Fold Cross-Validation, ensuring a model performs well on unseen data — a trustworthy process for analytics work — and finally the full model-building process, from importing data through running experiments (each with its own technique, parameters, and features), comparing results, and building and deploying the final model.

5.6 Review Questions

1. Explain how a model differs from an algorithm, with an example.
2. Write Python code implementing the following logistic regression model:

$$f(x, y, z) = \frac{1}{1 + e^{-(5+2x-3y+4z)}}$$

3. Suppose you're asked to analyze whether the Thai baht will rise or fall tomorrow. Explain what type of analytics problem this is.

4. After K-Fold Cross-Validation testing is complete, explain which round's model should be used in production, and why.
5. From the model-building process in section 5.4.2, explain which steps would need to change to use Train-Test Split instead, and how.

5.7 References

Bishop, C. M., & Nasrabadi, N. M. (2006). Pattern Recognition and Machine Learning (Vol. 4, No. 4). Springer.

Igual, L., & Seguí, S. (2017). Introduction to Data Science. Springer, Cham.

O'Neil, C., & Schutt, R. (2013). Doing Data Science: Straight Talk from the Frontline. O'Reilly Media, Inc.

Provost, F., & Fawcett, T. (2013). Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking. O'Reilly Media, Inc.